



Crucidex

Explore, share, and index your games

A Social Platform for Game Library Tracking and Discovery

Jess Barrett | Dr. Nathan Bean | CIS 598 Spring 2026

Site: crucidex.com

What is Crucidex

Inspiration: Crucidex is inspired by Letterboxd, a social platform for tracking movies watched, sharing reviews, and finding new movies.

One sentence pitch: Crucidex is a social network for cataloging the games you have played, sharing reviews, and discovering new titles through collaborative filtering.

Three core pillars: Track, Share, Discover

The Problem

- Gamers' libraries are scattered: Steam, Epic, Xbox, PlayStation, physical
- Existing trackers either don't exist or feel outdated
- No equivalent to Letterboxd for games — combining personal tracking + social + recommendations

Related Apps

- Letterboxd — gold standard for media tracking, inspired the UX
- Backloggd — closest direct competitor, has gaps (limited friend activity, weaker recommendations)
- Steam community — game library exists but no social aspect or review depth
- HowLongToBeat — playtime-focused, no library or social aspect

How Crucidex Differs

- Hybrid recommendation engine (most competitors use simple genre matching)
- Steam auto-import with intelligent IGDB matching
- Real-time activity feed via DB triggers (vs polling)
- Mutual friendships + open profiles

Functional Requirements

- User accounts (email auth, profiles, avatars, bios)
- Game library (add manually or via Steam, ratings, status, hours played, top 4)
- Reviews (write, read, link to library entries)
- Friends (mutual: send request → accept/decline)
- Activity feed (own + future: friends')
- Recommendation engine (personalized, dismissible)
- Wishlist
- Game detail pages (per-IGDB game stats, aggregate ratings)

Non-Functional Requirements

- Responsive design (desktop primary, mobile usable)
- Fast — quick tab navigation
- Secure — RLS on every table
- Scalable storage — handles hundreds of games per user
- Real-time-feeling activity (no manual refresh)

User Stories

- "As a Steam user, I want to import my library so I don't have to re-enter games"
- "As a frequent gamer, I want recommendations from people who like what I like"
- "As a returning user, I want to see what my friends played recently"

Tech Stack

Frontend:

- Next.js 16
- React 19
- Typescript
- Tailwind CSS v4

Backend:

- Supabase
 - ◆ Postgres
 - ◆ Auth
 - ◆ Storage
 - ◆ Row-Level Security

External APIs:

- IGDB (via Twitch OAuth)
- Steam Web API
- Steam OpenID

Infrastructure:

- Vercel (hosting)
- Resend (email)
- Github (version control)

Code Structure

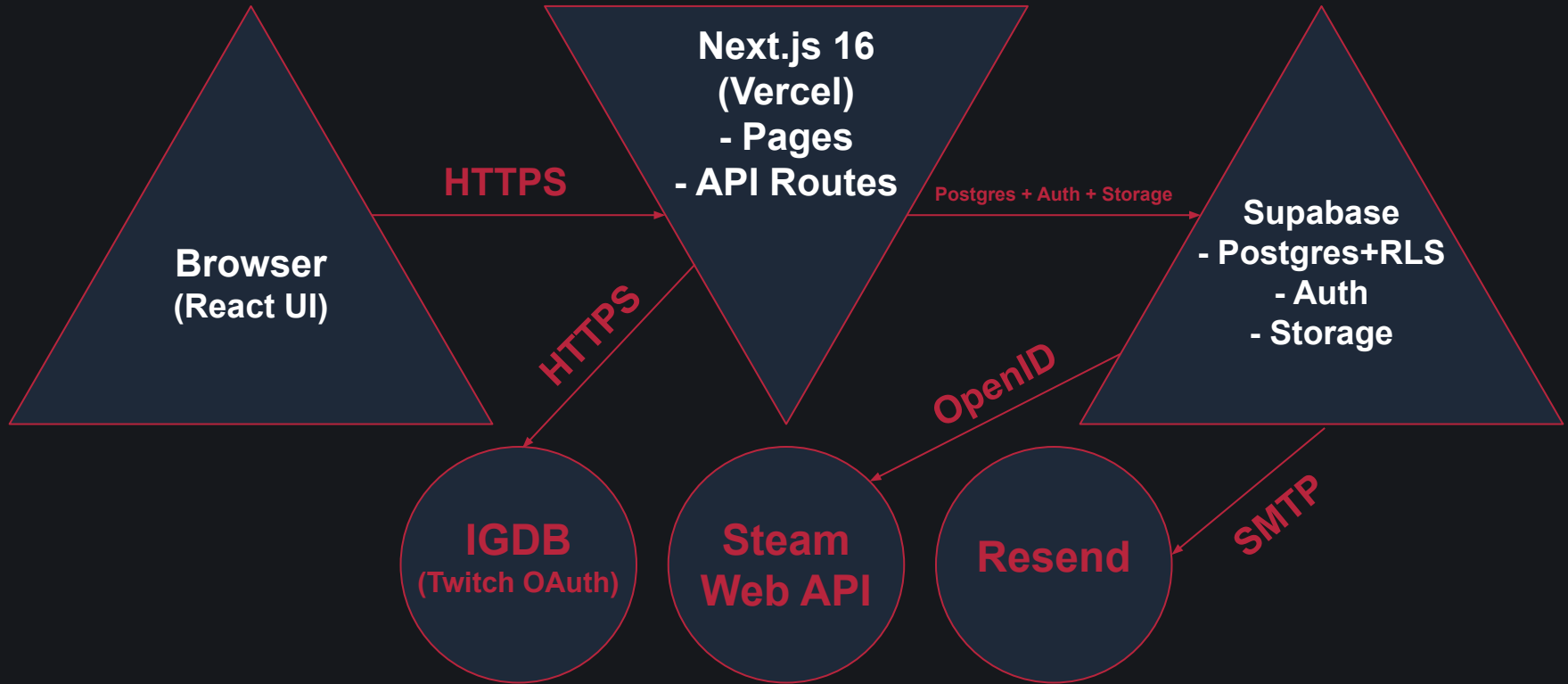
app/

— api/	Server-side API routes
— recommendations/	Hybrid recommendation engine
— activity/	Activity feed reader
— friends/	Friend request CRUD
— steam/	Steam OAuth + library import
— games/search/	IGDB-backed game search
— auth/	Auth callback + email confirm pages
— components/	Shared React components
— games/[igdb_id]/	Per-game detail page
— u/[username]/	User profile pages (layout-based tabs)
— settings/	User settings

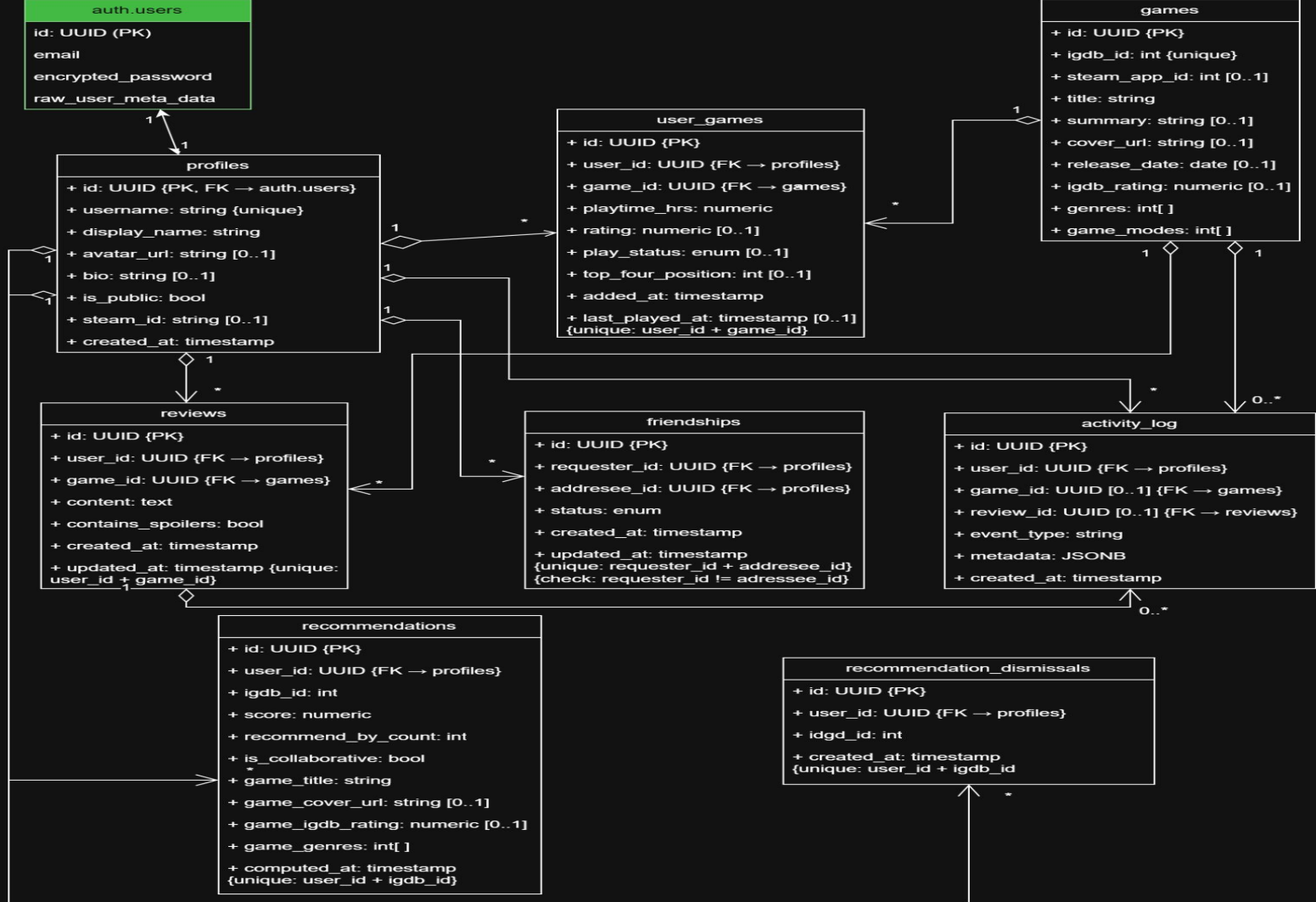
lib/

— supabase-client.ts	Browser Supabase client
— supabase-server.ts	Server-side Supabase client
— igdb.ts	IGDB API helpers + Steam matching
— game-stats.ts	Aggregate stats helpers
— profile-layout-context.tsx	Shared layout context

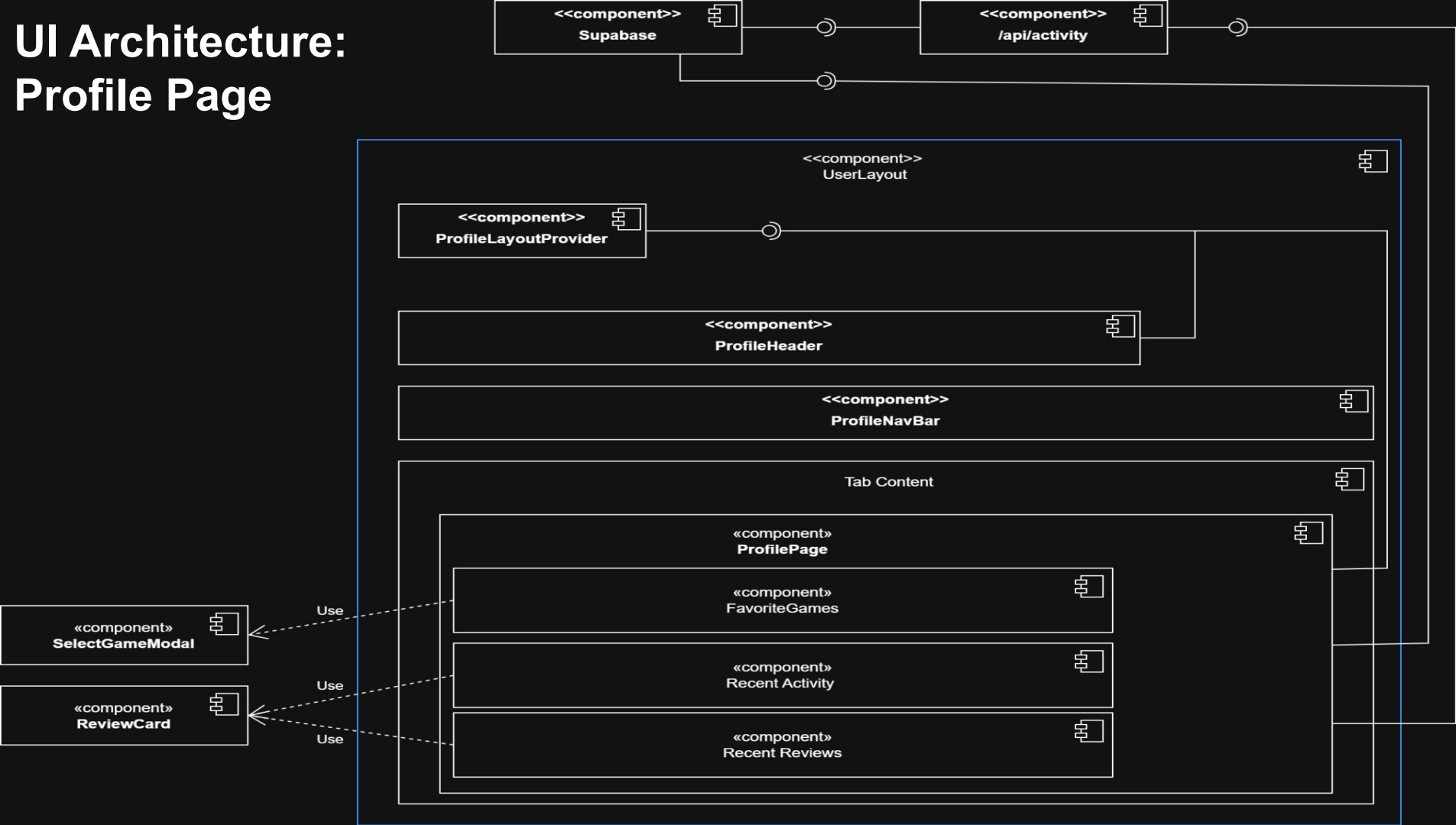
System Architecture



Database Schema

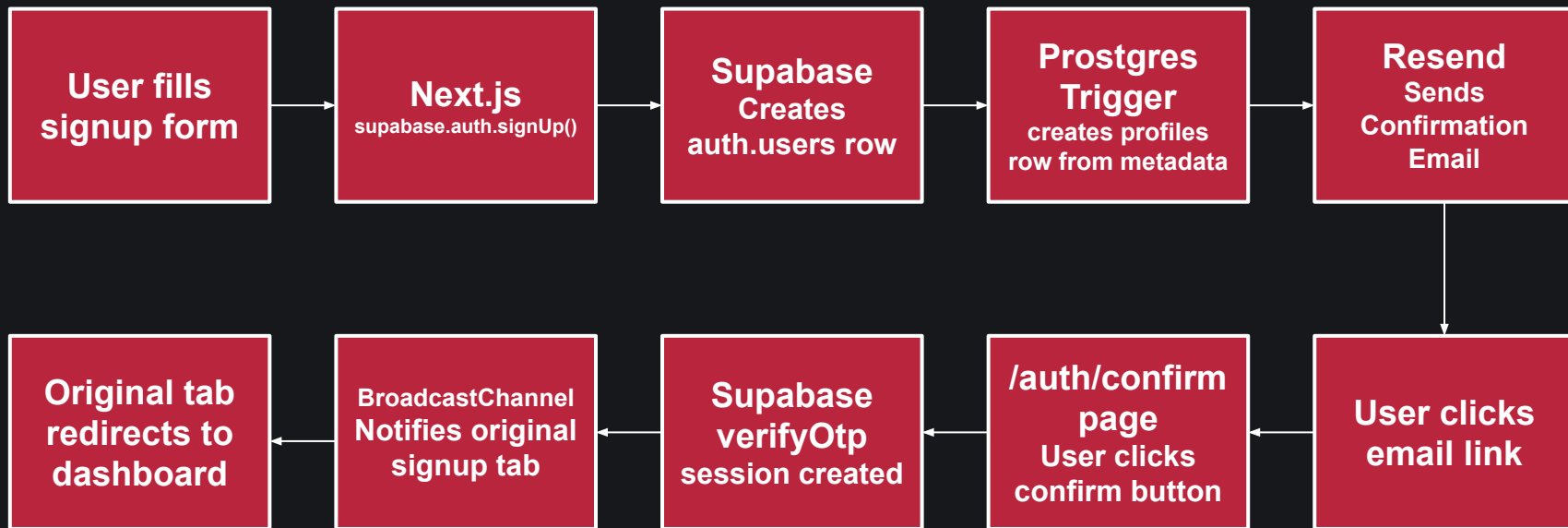


UI Architecture: Profile Page



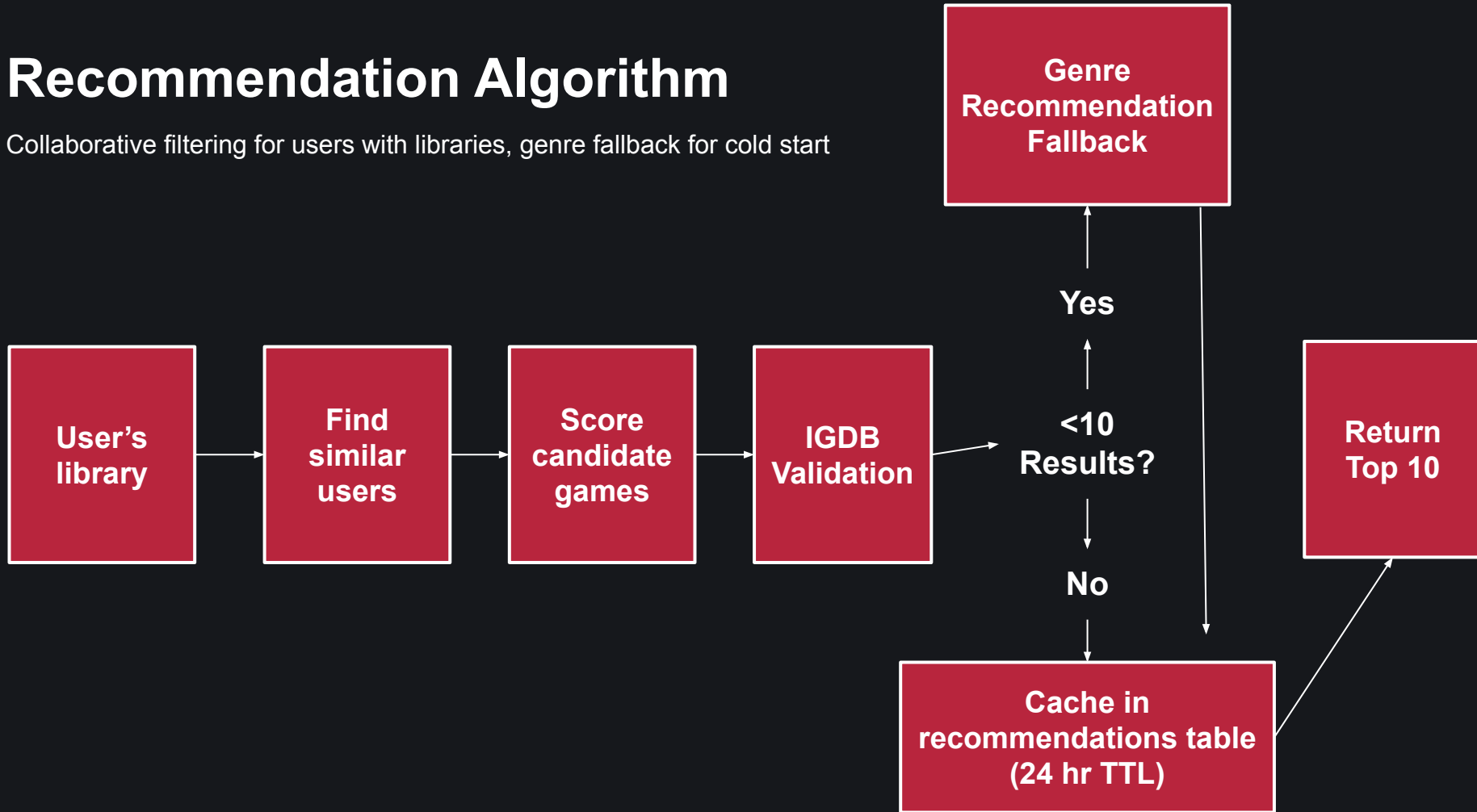
Authentication Flow

Email signup with custom two-step confirmation, plus optional Steam account linking.



Recommendation Algorithm

Collaborative filtering for users with libraries, genre fallback for cold start



Activity Log via Postgres Triggers

Naive - App Emits Events

App: addGameToLibrary()

- └ INSERT user_games
- └ INSERT activity_log

App: editRating()

- └ INSERT user_games
- └ INSERT activity_log

Steam import: bulk update

- └ UPDATE user_games
(forgot to log activity)

- ✗ Easy to miss
- ✗ Steam updates skip events
- ✗ Two writes per action

Trigger - Database Emits Events

App: INSERT INTO user_games

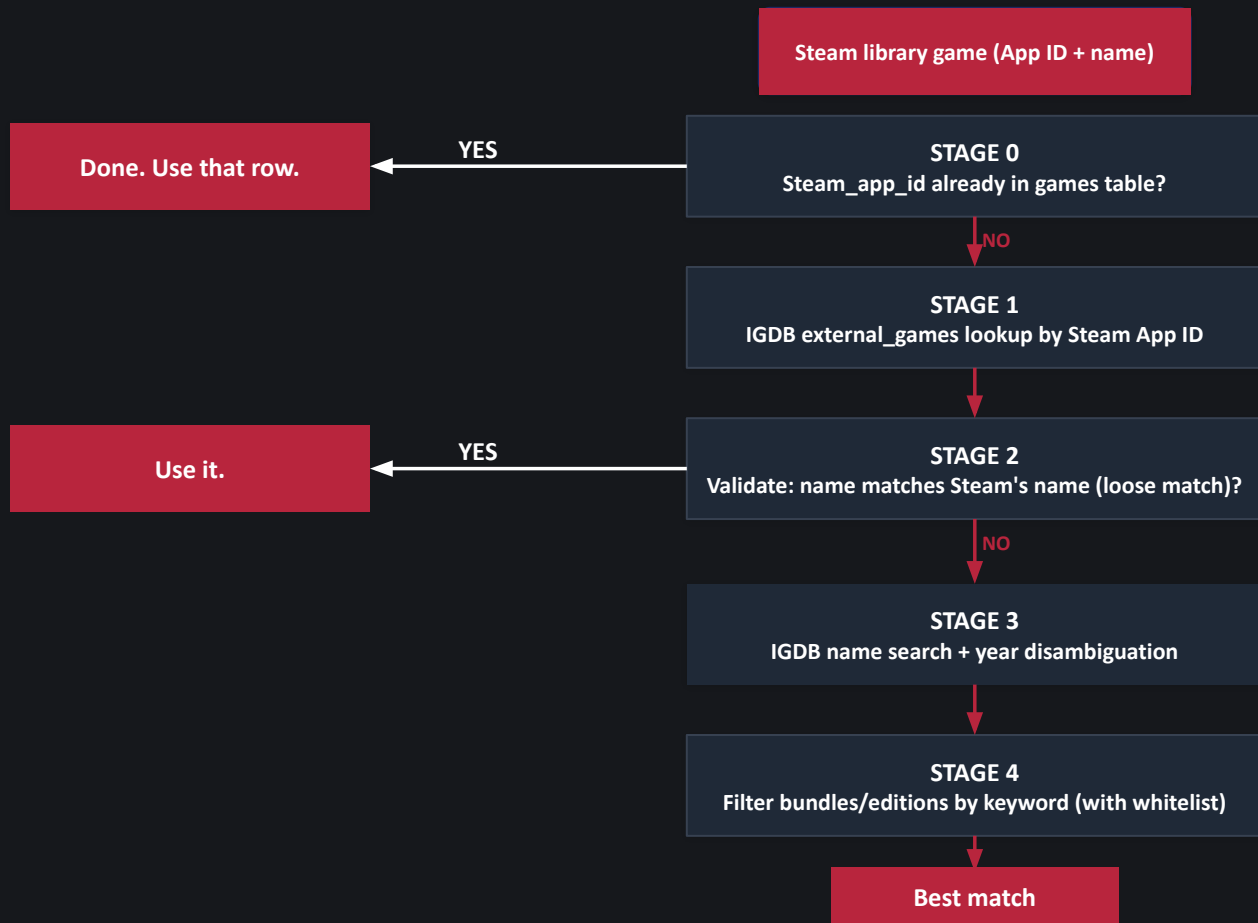
↓
Trigger fires
AFTER UPDATE
→ activity_log

App: UPDATE user_games

↓
Trigger fires
AFTER UPDATE
→ activity_log

- ✓ Single source of truth
- ✓ Steam updates auto-logged
- ✓ One write, trigger handles rest

Steam → IGDB Game Matching Pipeline



Testing & Evaluation

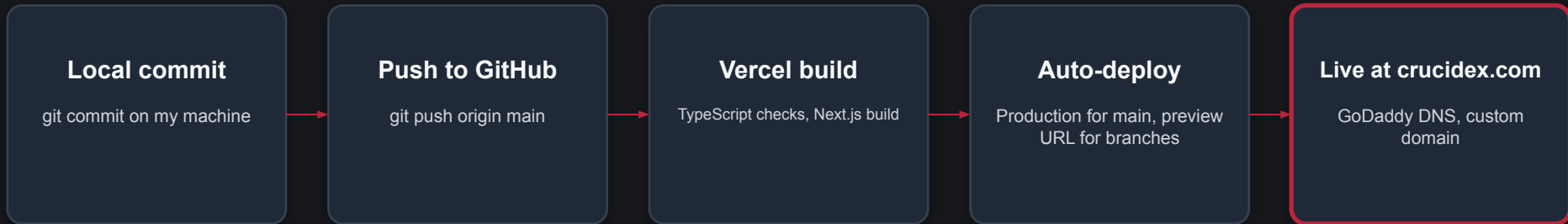
How I Tested

- **Manual Integration Testing through every feature flow**
- **Real Steam library imports with real data**
- **Edge cases such as large libraries, ambiguous game names, unconfirmed users, and expired links**

Who Tested It

- **Me, throughout development**
- **Classmates and friends imported their Steam libraries**
- **Test users with varying library sizes**

Deployment Pipeline



Database migrations are SQL files in the repo, applied by hand in Supabase.

Bugs Found & Fixed



Steam imports invisible

Problem Steam-imported games had NULL play_status. Library filter excluded NULLs.

Fix Include NULL explicitly in the filter.



Wrong avatar after upload

Problem Browser cached the old image at the same URL.

Fix Timestamped filenames so each upload is a new URL.



Duplicate recommendations

Problem Rapid “Not Interested” clicks each got the same replacement.

Fix Serialize requests + track known IDs in a ref.



Email links “expired”

Problem Corporate email scanners pre-fetched links, consuming the OTP before the user.

Fix Two-step confirmation page.

Performance

01

Layout persistence

Header stays mounted across tab switches — only the content area reloads.

02

Recommendation caching

Recommendations computed once and stored for 24 hours per user, instead of recomputed every page load.

03

Debounced live search

Game search waits 300ms after the user stops typing before hitting the API.

04

Optimistic UI updates

Dismissing a recommendation removes the card instantly, even before the server replies.

05

Cached game metadata

Game data from IGDB is stored in my own database after the first lookup, so we don't keep hitting the API.

Live Demo

crucidex.com

What's Next

Things I want to add as I keep building.



Public friend activity feed

See what your friends have been adding, rating, and reviewing — right on the dashboard.



Game-detail community pages

Per-game pages showing friends who've played, average ratings, and rating distributions.



Like and comment on reviews

Make reviews more social — let friends react and reply.



More library imports

Bring in games from Epic, GOG, Xbox, and PlayStation, not just Steam.



Mobile polish

Make the site feel native on phones, not just usable.



Year-in-review

A “wrapped” summary of each user's year — top games, hours played, genre breakdown.

What I Learned

A few takeaways from building this.

Postgres triggers as event sources

Letting the database emit events instead of the app turned out to be the cleanest pattern in the project.

Real-user testing catches what you can't predict

The bugs I'm most proud of fixing came from friends importing weird Steam libraries, not from my own testing.

Database schema is hard to change later

Decisions I made early about table shapes were the hardest things to undo. It pays to think carefully up front

Third-party APIs always have quirks

IGDB authenticates through Twitch. Steam matches games inconsistently. Email scanners pre-fetch links. The docs never warn you about any of it.

What I Built

A look at the project by the numbers.

~10,000

lines of TypeScript

30+

React components

12

API routes

9

database tables

Real users. Real Steam libraries. Live and running.

Thank You

Questions?

Live site: `crucidex.com`

GitHub: `github.com/jess-barrett/Crucidex`